УЧЕБНО-ПРОИЗВОДСТВЕННЫЙ ПЛАН

14 дней развития игры

От стабильного прототипа к автономной аварии и разумному экипажу

DX_FrameWork

Офлайн-версия · 16 августа 2026

Содержание

- 1. Как проходить план
- 2. Итоговая архитектура вертикального среза
- День 1. Защитная сетка: тестовая сцена и телеметрия
- День 2. Единое время симуляции и журнал событий
- День 3. Восприятие и память NPC
- День 4. World Task Board и reservation
- День 5. Тепло и пожар как самостоятельная симуляция
- День 6. Энергия, перегрузка и объяснимая поломка
- День 7. NPC сам выбирает и тушит пожар
- День 8. Атмосфера, здоровье и цена аварии
- День 9. Роли, навыки, усталость и травмы экипажа
- День 10. Автономный incident director
- День 11. Полноценное сохранение вертикального среза
- День 12. Цельный игровой сценарий
- День 13. Читаемость, звук, производительность и очистка
- День 14. Playtest, заморозка, документация и пакет на проверку
- 3. Что делать, если день застрял
- 4. Теоретические вопросы для самопроверки
- 5. Бэклог после двух недель
- 6. Минимальный офлайн-набор файлов

1. Как проходить план

Каждый день разделён на пять частей:

1. **Теория** — понять принцип до кода.
2. **Разведка** — найти связанные места в проекте.
3. **Реализация** — сделать небольшой законченный кусок.
4. **Проверка** — воспроизводимый тест, а не «вроде работает».
5. **Отчёт** — записать результат, чтобы после поездки или для моей проверки не восстанавливать контекст.

Ежедневный ритуал

В начале:

```
Сегодняшняя цель одной фразой:
Какие 3–5 файлов трогаю:
Как пойму, что готово:
Что не входит в задачу:
```

В конце добавь в `docs/DEV_NOTES_OFFLINE.md`:

```
Дата / день roadmap:
Что сделал:
Что компилируется:
Как воспроизвести:
Что сломано или непонятно:
Какие числа/логи получил:
Следующий маленький шаг:
```

Если времени мало

Выполни только пункты с меткой **Обязательно**. Не объединяй два дня в один большой хаотичный коммит.

Точка восстановления

Перед первым днём:

1. Собери `Debug x64`.
2. Проверь игрока и NPC на полу, в воздухе и на пороге.
3. Сохрани текущую рабочую сцену под новым именем.
4. Создай git-ветку, например `offline-two-week-slice`.
5. Сделай исходный коммит, если рабочее дерево подготовлено и ты понимаешь его состав.

2. Итоговая архитектура вертикального среза

```
Power/Heat simulation
        ↓
Fire or equipment failure
        ↓
World event + room alarm
        ↓
NPC perception/memory
        ↓
World task board
        ↓
Utility score + reservation
        ↓
NavMesh/doors/controller
        ↓
Extinguish/repair action
        ↓
Atmosphere, health, visuals
        ↓
Save/load + incident log
```

Игрок может помочь, проигнорировать или вообще находиться в другой части корабля.

День 1. Защитная сетка: тестовая сцена и телеметрия

Результат дня

У тебя есть отдельная regression-сцена и одна панель, которая показывает состояние физики персонажа, AI и симуляционного времени.

Теория

Сложный движок невозможно развивать, если каждое изменение проверяется случайной прогулкой по большой сцене. Регрессионная сцена — маленький набор геометрических и логических ситуаций, которые раньше ломались.

Наблюдаемость состоит из трёх уровней:

- числа в UI: скорость, grounded, state, target;
- линии в мире: ray, path, bounds, collider;
- события в журнале: «задача выбрана», «путь не построен».

Разведка

Открой:

- `src/Core/ApplicationClass.cpp`;
- `src/Editor/EditorUI.cpp`;
- `src/Gameplay/Player/CharacterControllerSystem.h`;
- `src/Gameplay/AI/NpcNavMeshSystem.cpp`;
- `src/Core/PhysicsWorld.h`.

Найди `m_stepIndex`, grounded, current/previous physics transform и debug status NPC.

Обязательно

- Создай/сохрани сцену `CharacterRegression.json`.
- Добавь ровный пол, пороги 0.05/0.10/0.20/0.30/0.36 м, дверь и склон.
- Поставь player и один NPC.
- В debug UI покажи:
- entity ID;
- `grounded`;
- capsule center Y;
- capsule bottom Y;
- ground hit Y;

- horizontal speed;
- physics step index;
- NPC state, path size и next point.

Пример диагностических данных

```
struct CharacterDebugSnapshot
{
    Entity entity = InvalidEntity;
    bool grounded = false;
    float capsuleCenterY = 0.0f;
    float capsuleBottomY = 0.0f;
    float groundY = 0.0f;
    float horizontalSpeed = 0.0f;
    uint64_t physicsStepIndex = 0;
};
```

Не обязательно сохранять это как постоянный ECS-компонент. Можно заполнять runtime snapshot в debug build.

Проверка

- Player стоит 60 секунд: позиция визуально стабильна.
- NPC стоит 60 секунд: стабилен.
- NPC следует ходьбой через каждый допустимый порог.
- При прыжке grounded меняется ожидаемо.
- Step index растёт только при реальном physics step.

Готово, если

Любой будущий баг «дрожит/не идёт» можно подтвердить числами за две минуты.

Что прислать на проверку

Скрин debug UI, имя сцены, значения стоящего персонажа и список изменённых файлов.

День 2. Единое время симуляции и журнал событий

Результат дня

Появляется `SimulationClock` и ограниченный кольцевой журнал мировых событий.

Теория

Есть три разных времени:

- frame time — для UI, камеры и отображения;
- fixed physics time — для Bullet;
- world simulation time — для пожара, голода, износа, задач и save/load.

Мировое время не должно зависеть от количества кадров. События должны иметь timestamp, иначе AI и сохранение не смогут объяснить порядок причин.

Обязательно

Создай данные часов:

```
struct SimulationClock
{
    double totalSeconds = 0.0;
    float timeScale = 1.0f;
    bool paused = false;

    void Advance(float realDt)
    {
        if (!paused)
            totalSeconds += static_cast<double>(realDt * timeScale);
    }
};
```

Создай тип события:

```
enum class WorldEventType
{
    EquipmentFailed,
    FireStarted,
    FireExtinguished,
    BreachOpened,
    BreachSealed,
    PowerBrownout
};

struct WorldEvent
{
    WorldEventType type{};
    double timeSeconds = 0.0;
    Entity source = InvalidEntity;
    Entity subject = InvalidEntity;
    Entity room = InvalidEntity;
    float severity = 0.0f;
};
```

Используй кольцевой буфер максимум на 256–1024 событий, чтобы debug-журнал не рос бесконечно.

Важно

Event log — история фактов. Он не обязан заменять компоненты текущего состояния. Пожар существует как `FireComponent`; событие `FireStarted` лишь сообщает, когда он возник.

Проверка

- пауза останавливает world time;
- изменение timeScale предсказуемо ускоряет его;
- одинаковый тест при 30 и 144 FPS даёт близкое totalSeconds;
- буфер не растёт после лимита;
- UI показывает последние 20 событий.

Готово, если

Можно ответить: «что произошло, где, с кем и в какое игровое время».

Что прислать

Код часов, тип события, скрин журнала и результат теста при двух FPS.

День 3. Восприятие и память NPC

Результат дня

NPC знает не весь мир, а только увиденное/услышанное/полученное от датчика.

Теория

Разделяй:

- **world truth** — реальное состояние;
- **observation** — единичное наблюдение;
- **memory** — сохранённое знание с возрастом и уверенностью;
- **decision** — действие на основании памяти.

Если utility напрямую перебирает все пожары в registry, NPC телепат. Это удобно для прототипа, но противоречит концепту.

Обязательно

```
enum class MemoryFactType
{
    Fire,
    Breach,
    InjuredCrew,
    BrokenEquipment
};

struct NpcMemoryFact
{
    MemoryFactType type{};
    Entity subject = InvalidEntity;
    Entity room = InvalidEntity;
    XMFLOAT3 lastKnownPosition{};
    double observedAt = 0.0;
    float confidence = 1.0f;
};

struct NpcMemoryComponent
{
    std::vector<NpcMemoryFact> facts;
};
```

Добавь систему, которая:

1. получает наблюдения в текущей комнате или от сигнализации;
2. обновляет существующий факт вместо дубля;
3. снижает confidence со временем;
4. удаляет слишком старые/невалидные факты;
5. ограничивает размер памяти.

Пример затухания

```
fact.confidence = std::max(
    0.0f,
    fact.confidence - confidenceDecayPerSecond * dt);
```

Для тревоги от автоматического датчика confidence может быть выше, чем для слабого звука.

Проверка

- NPC в другой закрытой комнате не знает о тихой поломке;
- при открытой двери/сигнале получает факт;
- факт обновляется, а не дублируется каждый кадр;
- уничтоженная цель удаляется из памяти;
- через заданное время confidence уменьшается.

Готово, если

Debug UI показывает не список всех мировых проблем, а собственную память выбранного NPC.

День 4. World Task Board и reservation

Результат дня

Мировые проблемы превращаются в задачи, которые можно занять, выполнить, отменить и освободить.

Теория

Проблема и задача — не одно и то же. Один пожар может породить задачи «тушить», «эвакуировать раненого», «закрыть дверь», «отключить питание».

Reservation предотвращает толпу агентов у одной цели. Lease/expiry освобождает задачу, если NPC погиб или застрял.

Обязательно

```
using WorldTaskId = uint64_t;

enum class WorldTaskState
{
    Available,
    Reserved,
    InProgress,
    Completed,
    Cancelled
};

struct WorldTask
{
    WorldTaskId id = 0;
    WorldTaskType type{};
    WorldTaskState state = WorldTaskState::Available;
    Entity target = InvalidEntity;
    Entity room = InvalidEntity;
    Entity reservedBy = InvalidEntity;
    double reservationExpiresAt = 0.0;
    float priority = 0.0f;
};
```

API доски:

```
WorldTaskId AddTask(const WorldTask& task);
bool TryReserve(WorldTaskId id, Entity npc, double now, double leaseSeconds);
bool RenewReservation(WorldTaskId id, Entity npc, double now);
void Release(WorldTaskId id, Entity npc);
void Complete(WorldTaskId id, Entity npc);
void CancelTasksForTarget(Entity target);
```

Инварианты

- одну задачу резервирует максимум один NPC;
- резервировать completed нельзя;
- release чужим NPC не работает;
- истёкший lease возвращает задачу в Available;

- уничтожение target отменяет задачи;
- duplicate prevention не создаёт одну и ту же задачу каждый tick.

Проверка

Сделай unit-like тест через debug-кнопку:

1. создать задачу;
2. NPC A резервирует — успех;
3. NPC B резервирует — отказ;
4. время проходит — lease истёк;
5. NPC B резервирует — успех;
6. target уничтожен — задача Cancelled.

Готово, если

В UI есть таблица задач и причина, почему агент не может взять задачу.

День 5. Тепло и пожар как самостоятельная симуляция

Результат дня

Устройство может нагреться и породить пожар; пожар растёт, потребляет кислород и может погаснуть.

Теория

Не пытайся сразу сделать физически точное горение. Нужна устойчивая системная модель:

```
growth = fuelFactor × oxygenFactor × baseGrowth
decay  = extinguishing + oxygenStarvation + noFuel
intensity += (growth - decay) × dt
```

Ключевое — причинность, ограничения и измеримость.

Обязательно

```
struct FireComponent
{
    Entity room = InvalidEntity;
    float intensity = 0.05f;       // 0..1
    float fuel = 10.0f;            // условные кг/единицы
    float oxygenUsePerSecond = 0.2f;
    float heatPerSecond = 15.0f;
    bool extinguished = false;
};

void FireSystem::Update(ECSRegistry& registry, float dt)
{
    registry.ForEach<FireComponent>(
        [&](Entity fireEntity, FireComponent& fire)
        {
            RoomComponent* room = registry.GetComponent<RoomComponent>(fire.room);
            if (!room || fire.fuel <= 0.0f)
            {
                fire.extinguished = true;
                return;
            }

            const float oxygenFactor = ComputeOxygenFactor(*room);
            const float growth = 0.04f * oxygenFactor * std::min(fire.fuel, 1.0f);
            fire.intensity = std::clamp(fire.intensity + growth * dt, 0.0f, 1.0f);
            fire.fuel = std::max(0.0f, fire.fuel - fire.intensity * 0.03f * dt);
            ConsumeRoomOxygen(*room, fire.oxygenUsePerSecond * fire.intensity * dt);
        });
}
```

Функции и поля комнаты адаптируй к текущей реализации.

Порядок обновления

Power → device heat/failure → fire → atmosphere → damage/vitals → effects.

Проверка

- при нормальном кислороде intensity растёт;

- без топлива пожар гаснет;
- при низком кислороде рост падает;
- значения не зависят заметно от FPS;
- `FireStarted` создаётся один раз;
- после тушения `FireExtinguished` создаётся один раз.

Не делать сегодня

Красивый shader огня, сложный дым, распространение по всей станции и NPC.

День 6. Энергия, перегрузка и объяснимая поломка

Результат дня

Пожар/отказ возникает из состояния оборудования и сети, а не по таймеру рядом с игроком.

Теория

Свяжи уже существующие `GeneratorComponent`, `PowerConsumerComponent`, `PowerGridComponent` с теплом.

Причинная цепочка:

```
consumer получает MW
→ генерирует тепло
→ охлаждение не справляется
→ overheated
→ wear/failure chance растёт
→ equipment failed / fire started
```

Вероятность должна считаться на единицу времени, иначе FPS меняет частоту аварий.

Пример hazard rate

```
const float chancePerSecond = baseFailureRate * overheatMultiplier;
const float chanceThisFrame = 1.0f - std::exp(-chancePerSecond * dt);

if (random01 < chanceThisFrame)
    FailEquipment(entity);
```

Для воспроизводимости используй отдельный seed симуляции.

Обязательно

- Добавь `DeviceHeatComponent` минимум одному реальному power consumer.
- HeatSystem читает фактически полученную мощность.
- При пороге устройство выключается или создаёт failure.
- Failure создаёт `WorldEvent`.
- При подходящих условиях создаётся `FireComponent` и задача/сигнал.
- Debug UI показывает: requested/actual power, temperature, thresholds, failure cause.

Проверка

- без питания устройство остывает;
- нормальная нагрузка стабилизируется ниже safe threshold;
- перегрузка повышает температуру;
- save/load сохраняет температуру и broken state;
- одинаковый seed воспроизводит последовательность.

Готово, если

Ты можешь по журналу доказать, почему начался конкретный пожар.

День 7. NPC сам выбирает и тушит пожар

Результат дня

NPC получает знание, оценивает задачу, резервирует её, приходит через двери и тушит пожар.

Теория utility

Score должен отвечать на вопрос «насколько это действие подходит этому NPC сейчас».

```
score = urgency
      × roleSuitability
      × toolFactor
      × pathFactor
      × safetyFactor
      × distanceFactor;
```

Если любой обязательный фактор равен нулю, задача недоступна.

Обязательно

- Расширь command/state для `ExtinguishFire`.
- Utility смотрит на память NPC или task board, а не на все entities.
- Проверь роль/инструмент.
- Зарезервируй task перед движением.
- Используй существующий NavMesh/door pipeline.
- На дистанции тушения останови movement.
- Уменьшай fire intensity и расходуй огнетушитель.
- Освобождай reservation при success/cancel/blocked/death.

Псевдокод state

```
case NpcState::MoveToFire:
    if (!IsTaskValid(task))
        CancelCurrentTask(brain);
    else if (IsWithinExtinguishDistance(npc, fire))
        brain.state = NpcState::Extinguishing;
    else
        FollowCurrentPath(brain, movement);
    break;

case NpcState::Extinguishing:
    StopMovement(movement);
    ApplyExtinguishing(fire, tool, dt);
    RenewReservation(task, npc, now);
    if (fire.extinguished)
        CompleteTask(task, npc);
    break;
```

Hysteresis

Не переключай MoveToFire/Extinguishing на одной дистанции.

```
enterActionDistance = 1.6f;
leaveActionDistance = 2.0f;
```

Проверка

- NPC идёт ходьбой через знакомый порог;
- открывает дверь;
- не дрожит рядом с точкой действия;
- второй NPC выбирает другую задачу или ждёт;
- потеря пути переводит задачу в понятный blocked/cancel flow;
- отсутствие огнетушителя даёт причину отказа;
- игрок может погасить пожар раньше — NPC корректно отменяется.

Итог первой недели

Сделай 10-минутный playtest без изменения кода. Запиши все случаи, когда AI ведёт себя необъяснимо.

День 8. Атмосфера, здоровье и цена аварии

Результат дня

Пожар влияет на комнату и обычных персонажей: кислород падает, температура/опасность растёт, игрок и NPC получают последствия.

Теория

Связь систем должна идти через данные комнаты:

```
FireSystem меняет room atmosphere
AtmosphereSystem распространяет значения
Vitals/DamageSystem читает room atmosphere
AI perception читает alarm/hazard
EffectsSystem только отображает
```

Обязательно

- Огонь потребляет кислород комнаты.
- Опасная среда наносит постепенный урон или снижает эффективность.
- Дверь влияет на обмен между комнатами.
- NPC учитывает опасность в utility.
- Player не получает иммунитет.
- Добавь понятный HUD/debug индикатор кислорода и опасности текущей комнаты.

Пример доза–эффект

```
float ComputeHazardDamage(float oxygenFraction, float temperatureC)
{
    const float hypoxia = std::max(0.0f, 0.16f - oxygenFraction) * 40.0f;
    const float heat = std::max(0.0f, temperatureC - 55.0f) * 0.03f;
    return hypoxia + heat;
}
```

Это игровые стартовые числа, не медицинская модель.

Проверка

- закрытая дверь локализует последствия;
- открытая дверь распространяет их;
- огонь не потребляет отрицательный кислород;
- player и NPC получают одинаковые базовые правила;
- мир работает, когда player далеко.

День 9. Роли, навыки, усталость и травмы экипажа

Результат дня

Два NPC ведут себя по-разному по объяснимым причинам.

Теория

Хороший агент — не тот, у кого огромный behaviour tree, а тот, у кого есть ограничения и контекст.

```
enum class CrewRole { Engineer, Medic, Security, Generalist };

struct CrewSkillsComponent
{
    CrewRole role = CrewRole::Generalist;
    float engineering = 0.2f;
    float medicine = 0.2f;
    float endurance = 0.5f;
};

struct CrewNeedsComponent
{
    float fatigue = 0.0f;
    float stress = 0.0f;
    float pain = 0.0f;
};
```

Обязательно

- Инженер получает высокий score ремонта/тушения.
- Уставший NPC движется или работает медленнее.
- Раненый NPC оценивает риск выше.
- Навык меняет скорость/качество, но не создаёт магическую невозможность для всех остальных.
- UI показывает полный разбор score по факторам.

Пример скорости работы

```
const float fatigueFactor = std::clamp(1.0f - needs.fatigue * 0.6f, 0.25f, 1.0f);
const float skillFactor = 0.5f + skills.engineering;
const float workRate = baseRate * fatigueFactor * skillFactor;
```

Проверка

- инженер выигрывает равную задачу у generalist;
- ближайший generalist может выиграть срочную задачу у далёкого инженера;
- крайняя усталость не даёт NaN/отрицательную скорость;
- после отдыха эффективность восстанавливается.

День 10. Автономный incident director

Результат дня

Система может запустить один объяснимый инцидент на основании состояния мира и seed.

Теория

Director не должен «развлекать игрока» постоянными атаками. Он должен управлять частотой проверки и ограничивать хаос, сохраняя причинность.

Состояния:

```
enum class IncidentPhase
{
    Quiet,
    Warning,
    Active,
    Recovery
};
```

Обязательно

- У director есть cooldown и максимальное число активных инцидентов.
- Кандидаты выбираются среди реально изношенного/перегретого оборудования.
- Комната игрока не получает специальный приоритет.
- Перед инцидентом возможен warning: шум, температура, нестабильность.
- Все решения пишутся в журнал вместе с причиной и seed.

Пример кандидата

```
struct IncidentCandidate
{
    Entity entity = InvalidEntity;
    float wearScore = 0.0f;
    float heatScore = 0.0f;
    float maintenanceNeglect = 0.0f;

    float Total() const
    {
        return wearScore * 0.4f + heatScore * 0.4f + maintenanceNeglect * 0.2f;
    }
};
```

Проверка

- при одинаковом seed сценарий воспроизводится;
- без кандидатов авария не создаётся;
- активный инцидент не размножается каждый tick;
- player может быть в другой комнате;

- recovery возвращает director в Quiet.

День 11. Полноценное сохранение вертикального среза

Результат дня

Можно сохранить аварию в середине, перезапустить игру и продолжить с тем же состоянием.

Теория

Сохраняй авторитетные данные:

- world time и random seed/state;
- heat/fire;
- atmosphere;
- состояние power и устройств;
- задачи и reservation либо правила их безопасного восстановления;
- memory NPC;
- роль, needs и current task;
- двери и пробоины.

Runtime-ресурсы пересоздаются.

Обязательно

- Добавь `saveVersion`.
- Новые поля загружаются с defaults.
- Entity references проходят remap.
- Запись идёт во временный файл, затем заменяет основной.
- Сделай минимум одну миграцию/default-проверку для старой сцены.

Решение по runtime-задачам

Выбери и документируй один вариант:

1. сохранять задачи и валидировать их после load;
2. не сохранять task board, а пересоздавать задачи из world state.

Для первой версии вариант 2 часто безопаснее: пожар после load снова публикует задачу, а NPC заново выбирает её.

Проверка save/load

Сохрани в четырёх состояниях:

- до аварии;
- пожар растёт;
- NPC идёт к пожару;
- пожар почти потушен.

После load проверь отсутствие дублей событий/задач и валидность ссылок.

Готово, если

Ручной reload не меняет причинную историю инцидента.

День 12. Цельный игровой сценарий

Результат дня

Есть 10–15-минутный сценарий, который можно пройти разными способами.

Сценарий «Ночной перегруз»

Начальные условия:

- корабль в пути;
- два NPC: инженер и generalist;
- один генератор работает близко к пределу;
- кислородный генератор и дверь питаются от сети;
- в технической комнате устройство постепенно перегревается;
- огнетушитель доступен, но не в руках NPC.

Возможные развития:

- NPC замечает предупреждение и предотвращает пожар;
- пожар начинается, инженер тушит;
- дверь обесточивается и путь меняется;
- player помогает;
- player игнорирует — экипаж справляется или терпит последствия;
- травма/нехватка инструмента делает задачу сложнее.

Обязательно

- Убери прямые debug-команды из нормального сценария.
- Настрой начальные данные через компоненты сцены.
- Добавь понятные сигналы: лампа, журнал, частицы или HUD.
- Запиши ожидаемую временную линию, но разреши отклонения AI.

Проверка

Пройди сценарий трижды:

1. активно помогая;
2. ничего не делая;
3. мешая системе: закрывая дверь/выключая питание.

Запиши, почему каждый исход произошёл.

День 13. Читаемость, звук, производительность и очистка

Результат дня

Система понятна игроку и не создаёт очевидных утечек/скачков нагрузки.

Теория

Игроку не нужны все числа debug UI. Ему нужны сигналы:

- что изменилось;
- насколько опасно;
- где проблема;
- что можно сделать;
- почему действие не работает.

Обязательно

- Отдели player-facing сообщения от debug-журнала.
- Ограничь частоту повторяющихся уведомлений.
- Проверь число task/event/fire entities после 20 минут.
- Измерь время новых систем.
- Убедись, что NavMesh не перестраивается и repath не вызывается каждый кадр.
- Удали мёртвый экспериментальный код только если точно понимаешь, что он не используется; иначе оставь с TODO.

Простой профилировочный scope

```
class ScopedTimer
{
public:
    explicit ScopedTimer(const char* name);
    ~ScopedTimer();
private:
    const char* m_name;
    std::chrono::high_resolution_clock::time_point m_start;
};

void FireSystem::Update(ECSRegistry& registry, float dt)
{
    ScopedTimer timer("FireSystem");
    // ...
}
```

Используй существующий profiler, если он уже есть, вместо второго параллельного инструмента.

Практикум: тяжёлый GLB без зависания редактора

В проект уже добавлены два асинхронных пути: `AssetLoadSystem` для обычной статической модели и staged import в `SceneManager` для корабля. Твоя задача сегодня — не переписать их, а доказать измерениями, что они работают, и найти следующий узкий участок.

Сначала зафиксируй четыре разных времени:

```
struct AssetLoadTimings
{
    double assimpReadMs = 0.0;       // worker
    double cpuPrepareMs = 0.0;       // worker
    double worstNodeUploadMs = 0.0;  // main thread
    double totalMs = 0.0;            // wall clock
};
```

`totalMs` может быть большим — это не ошибка. Важнее, чтобы главный поток продолжал выдавать кадры. Например, загрузка длится 40 секунд, но камера и ImGui отвечают: пользователь видит процесс и может принять решение. Настоящий freeze — отсутствие кадров и событий окна.

Порядок эксперимента:

1. Собери Debug x64 и запиши размер `assets/model/SpaceShip.glb`.
2. Запусти чистую сцену, нажми `SpawnShip` один раз.
3. Запиши время каждой стадии панели.
4. Во время `Reading GLB` двигай камеру и раскрывай UI.
5. Во время `UploadingNodes` следи за самым длинным кадром.
6. После `Ready` проверь двери, комнаты, collision proxies, панели и NPC path.
7. Повтори с несуществующим путём и зафиксируй понятный `Error`.
8. Загрузи обычный GLB через `Spawn` без `Spawn with physics` и проверь замену fallback mesh.

Не делай Direct3D-вызовы из worker ради красивых цифр. Безопасная граница остаётся такой:

```
// Worker: только CPU и неизменяемые входные данные.
jobs.SubmitJob([sharedState]
{
    sharedState->ReadAndPrepareWithAssimp();
    sharedState->phase.store(Phase::CpuReady, std::memory_order_release);
});

// Главный кадр: ECS, Bullet, Direct3D.
jobs.ExecuteMainThreadJobs();
assets.Update(device);
scene.UpdateAsyncShipImport(device, context, 4.0f);
```

Если один узел занимает больше 8–12 мс, найди его имя по `Ship node:` и запиши число вершин/индексов. Не оптимизируй вслепую. Возможные следующие шаги:

- вынести преобразование вершин каждого ship node в worker;
- кэшировать повторно используемые embedded textures;
- делить гигантский узел на chunks;
- строить тяжёлые collision-данные заранее;
- добавить `submesh/material ranges` обычному async GLB.

Минимальный отчёт по практикуму:

```
Файл и размер:
Assimp read:
CPU prepare:
Полное время:
Худший main-thread кадр:
Имя самого тяжёлого узла:
Окно отвечало во время чтения: да/нет
Что проверено после Ready:
Следующее узкое место:
```

Проверка

- 20 минут симуляции без постоянного роста контейнеров;
- уведомления не спамят;
- debug UI всё ещё объясняет причины;
- обычный HUD не показывает внутренний мусор;
- тяжёлый GLB показывает стадии и не блокирует события окна;
- повторный `SpawnShip` во время загрузки недоступен;
- ошибка пути видна в панели, а не превращается в crash;
- нет новых warning/error в Output.

День 14. Playtest, заморозка, документация и пакет на проверку

Результат дня

Вертикальный срез зафиксирован, воспроизводим и готов к ревью после возвращения связи.

Обязательно

1. Собери `Debug x64` с нуля.
2. Если проект поддерживает Release — собери `Release x64`.
3. Пройди CharacterRegression.
4. Пройди сценарий «Ночной перегруз» тремя способами.
5. Проверь save/load.
6. Проверь запуск после перезагрузки ПК.
7. Обнови этот документ фактическими именами новых файлов.
8. Заполни итоговый отчёт.
9. Сделай понятные коммиты или один финальный коммит с перечислением уже чистых изменений.
10. Сохрани резервную копию сцены и save.

Итоговый отчёт

```
Ветка / commit:
Конфигурация сборки:
Новые компоненты:
Новые системы:
Где они вызываются в ApplicationClass:
Новые JSON-поля и saveVersion:
Сценарий запуска:
Шаги проверки:
Ожидаемый результат:
Известные проблемы:
Логи/скриншоты:
Три вопроса на ревью:
```

Финальная Definition of Done

- пожар/отказ имеет системную причину;
- событие происходит без player proximity trigger;
- NPC узнаёт через perception/alarm;
- task резервируется одним NPC;
- агент проходит дверь и порог обычной ходьбой;
- рядом с целью нет дрожания;
- действие меняет реальный FireComponent;
- атмосфера и персонажи получают последствия;

- save/load сохраняет или корректно перестраивает состояние;
- журнал объясняет причинную цепочку;
- тест воспроизводим по инструкции.

3. Что делать, если день застрял

Правило 30 минут

Если 30 минут меняешь числа без новой информации, остановись и добавь измерение: лог, debug line или значение в UI.

Сужение задачи

Вместо «AI не работает» выясни:

```
Есть ли world problem?
Создано ли событие?
Есть ли факт в памяти NPC?
Создана ли task?
Может ли NPC её зарезервировать?
Какой utility score?
Есть ли path?
Заполнен ли CharacterMovement?
Сдвинулась ли capsule?
Синхронизирован ли Transform?
```

Первый ответ «нет» почти всегда показывает слой ошибки.

Если код перестал компилироваться

Не добавляй новую функцию. Вернись к первой ошибке, исправь build, затем продолжай. Никогда не оставляй проект некомпилируемым на ночь, если можешь этого избежать.

Если реализация слишком велика

Сохрани интерфейс и сделай простейшую версию. Например, perception сначала работает только внутри одной RoomComponent; line of sight добавишь позже.

4. Теоретические вопросы для самопроверки

Ответь своими словами, не подглядывая в код.

1. Почему Component не должен владеть большой игровой логикой?
2. Чем frame update отличается от fixed update?
3. Почему render interpolation должна продвигаться по факту physics step?
4. Почему нельзя поднять только capsule, не учитывая mesh offset?
5. Что означает `Detour could not project start/target`?

6. Зачем utility AI hysteresis?
7. Чем факт мира отличается от памяти NPC?
8. Почему task reservation нуждается в expiry?
9. Почему вероятность «раз в кадр» зависит от FPS?
10. Почему эффекты не должны быть источником gameplay state?
11. Зачем Entity remap после загрузки?
12. Почему Bullet/D3D-указатели нельзя записать в JSON?
13. Что является авторитетом transform физического персонажа?
14. Как доказать, что атмосфера не создаёт кислород из ничего?
15. Что делает мир автономным, а не вращающимся вокруг игрока?

Если не можешь ответить, открой соответствующий раздел полного справочника.

5. Бэклог после двух недель

Не начинай эти пункты до завершения вертикального среза.

Высокий приоритет

- обмен знаниями между NPC;
- перенос и reservation физических инструментов;
- лечение и спасение раненых;
- граф обслуживания оборудования;
- полноценный runtime save с миграциями;
- тесты сохранения массы/кислорода;
- LOD частоты симуляции далёких комнат.

Средний приоритет

- расписание смен и отдых;
- доверие, стресс и паника;
- логистика запчастей;
- несколько типов пожара;
- дым и ухудшение зрения;
- саботаж/ошибка экипажа как редкие системные причины;
- аудио-сигналы помещений.

Позже

- экономика и торговля;

- фракции и долгосрочные отношения;
- процедурные маршруты;
- космические станции и внешние EVA-задачи;
- крупные сюжетные цепочки, которые возникают из состояния мира.

6. Минимальный офлайн-набор файлов

Перед поездкой держи рядом:

- `DX_FRAMEWORK_FULL_DOCUMENTATION_RU.md` и `.docx`;
- этот roadmap в `.md` и `.docx`;
- `CODE_METHOD_MAP.md`;
- `CPP_ENGINE_THEORY_OFFLINE.md`;
- `NPC_AI_DEBUG_GUIDE_RU.md`;
- `PARTICLES_EXPLOSIONS_DECOMPRESSION_GUIDE_RU.md`;
- `MeshCuttingGuide.md`;
- `DEV_NOTES_OFFLINE.md`;
- копию solution, assets и тестовых сцен.

Конец roadmap.